Homework 5 - Tidy and Process the Billboard Dataset

Shivali Mate

The goal of this homework is to **tidy** the Billboard dataset and perform some basic analysis and visualization.

You will demonstrate your understanding of data tidying principles and your ability to manipulate and visualize data using Python.

The Billboard dataset comes with **76 columns** corresponding to the chart position of each song from `x1st.week` through `x76th.week`. This is a classic example of **wide** data that needs to be **melted** (unpivoted) into a long (tidy) format.

Instructions

1. **Project Setup**: Follow the instructions on how to set up your Python and Jupyter (or VSCode) environment, as well as how to clone or download our repository. These can be found in the class notes.
2. **Dataset Path**: Verify the path to the CSV file (e.g. `"../../Datasets/billboard.csv"`). Adjust if necessary.
3. **Load** the Billboard CSV into a DataFrame with `pd.read_csv()`.
4. **Melt/Unpivot** the 76 weekly columns (`x1st.week` through `x76th.week`) into two new columns:
   - `week` (indicating which week of the chart it is, e.g., 1, 2, 3...)
   - `rank` (the song's chart position for that week).
   - This is typically done using `pd.melt()` or `pd.wide_to_long()`.
5. **Clean** the `week` values so that they are numeric (1 to 76) instead of strings like `"x1st.week"`.
6. **Create a `date` column** that indicates the exact date on the chart for each row, using:

   ```
   date = date.entered + (week - 1) * 7 days
   ```

   - You may want to transform `date.entered` in a `datetime` format (you may need to convert it with `pd.to_datetime()` check documentation).
7. **Split** the data into two tables:
   1. **songs** table, which should contain unique song information (e.g., `song_id`, `artist`, `track`, `time`, etc.).
      - Remember to drop duplicates so you only have one row per song.
      - Generate a new `song_id` column that can be used to join back to the weekly info.
   2. **positions** table, which should contain weekly data: `(song_id, week, rank, date)`.
8. **Save** your tidy DataFrames to Feather format:
   - Save the full "tidy" DataFrame (one row per song-week) with a filename suffix `_tidy`.
   - Also save the separate `songs` and `positions` tables as separate Feather files (e.g., `songs.feather`, `positions.feather`).
9. **Submission**:
   - Export your completed notebook to **HTML** or **PDF**.
   - If on Jupyter, select `File > Export Notebook As > HTML`.
   - If on VSCode, use `Jupyter: Export to HTML` from the command palette.
   - Submit your exported file on Canvas.

Dataset Overview

The dataset consists of songs and their weekly chart positions on the Billboard Hot 100. It has the following main columns:

- `year`: The year the song entered the chart.
- `artist`: The artist of the song.
- `track`: The title of the song.
- `time`: The duration of the song.
- `date.entered`: The date the song first entered the chart.
- `x1st.week` to `x76th.week`: The chart position of the song for each of the 76 weeks (1 = highest rank, 100 = lower rank).

Goals (Detailed)

1. **Load** the Billboard dataset from CSV into a pandas DataFrame.
2. **Tidy** the data by converting the weekly columns into a single `week` column (numeric) and a `rank` column.
3. **Compute** an exact `date` for each row by adding `(week - 1) * 7 days` to the `date.entered`.
4. **Split** the data into:
   - **songs**: (One row per song) containing static info and a new `song_id`.
   - **positions**: (One row per song-week) containing columns: `[song_id, week, rank, date]`.
5. **Save** everything to Feather files (or CSV if you prefer, but Feather is recommended).

```
import pandas as pd

# 1. Load the Billboard dataset
df_bill = pd.read_csv("billboard.csv")  # Adjust path if needed

# Let's quickly check the first few rows to confirm the structure.
df_bill.head()
```

	year	artist.inverted	track	time	genre	date.entered	date.peaked	x1st.week	x2nd.week	x3rd.week	...	x67th.week	x68th.wee
0	2000	Destiny's Child	Independent Women Part I	3:38	Rock	2000-09-23	2000-11-18	78	63.0	49.0	...	NaN	Na
1	2000	Santana	Maria, Maria	4:18	Rock	2000-02-12	2000-04-08	15	8.0	6.0	...	NaN	Na
2	2000	Savage Garden	I Knew I Loved You	4:07	Rock	1999-10-23	2000-01-29	71	48.0	43.0	...	NaN	Na
3	2000	Madonna	Music	3:45	Rock	2000-08-12	2000-09-16	41	23.0	18.0	...	NaN	Na
4	2000	Aguilera, Christina	Come On Over Baby (All I Want Is You)	3:38	Rock	2000-08-05	2000-10-14	57	47.0	45.0	...	NaN	Na

5 rows × 83 columns

The dataset has columns like:

- **year**, **artist.inverted**, **track**, **time**, **genre**, etc. (Song info)
- **date.entered**, **date.peaked** (Chart-related dates)
- **x1st.week** through **x76th.week** (Chart positions over 76 weeks)

Our objective is to **melt** these weekly columns into a single `week` and `rank` column.

```
# 2. Melt/unpivot the data into a "tidy" format.
# HINT: you can use pd.melt().
# We'll create a new DataFrame, for example df_tidy.

# Your code here
df_tidy = df_bill.melt(
    id_vars=["year", "artist.inverted", "track", "time", "genre", "date.entered", "date.peaked"],
    var_name="week",
    value_name="rank"
)
```

After melting, each row should represent **one song** in **one week**. However, the `week` column will have strings like `"x1st.week"`, `"x2nd.week"`, etc. Let's clean those up and create a numeric week column.

```
# 3. Convert the week column to numeric (strip the "x", remove ".week", etc.)

# Your code here
df_tidy['week'] = df_tidy['week'].str.extract(r'(\d+)').astype(int)
df_tidy.head()
```

	year	artist.inverted	track	time	genre	date.entered	date.peaked	week	rank
0	2000	Destiny's Child	Independent Women Part I	3:38	Rock	2000-09-23	2000-11-18	1	78.0
1	2000	Santana	Maria, Maria	4:18	Rock	2000-02-12	2000-04-08	1	15.0
2	2000	Savage Garden	I Knew I Loved You	4:07	Rock	1999-10-23	2000-01-29	1	71.0
3	2000	Madonna	Music	3:45	Rock	2000-08-12	2000-09-16	1	41.0
4	2000	Aguilera, Christina	Come On Over Baby (All I Want Is You)	3:38	Rock	2000-08-05	2000-10-14	1	57.0

Next steps: Generate code with df_tidy | View recommended plots | New interactive sheet

Now, `week` should be 1, 2, 3, ... 76. Next, we want to calculate the **exact date** on the chart for each row by adding `(week - 1) * 7 days` to `date.entered`. Create a column named `"date"` to hold the result. Make sure `date.entered` is converted to datetime first, if it isn't already.

```
# 4. Create the 'date' column: date = date.entered + (week - 1)*7 days

# Your code here
df_tidy['date.entered'] = pd.to_datetime(df_tidy['date.entered'])
df_tidy['date'] = df_tidy['date.entered'] + pd.to_timedelta((df_tidy['week'] - 1) * 7, unit='days')
df_tidy.head()
```

	year	artist.inverted	track	time	genre	date.entered	date.peaked	week	rank	date
0	2000	Destiny's Child	Independent Women Part I	3:38	Rock	2000-09-23	2000-11-18	1	78.0	2000-09-23
1	2000	Santana	Maria, Maria	4:18	Rock	2000-02-12	2000-04-08	1	15.0	2000-02-12
2	2000	Savage Garden	I Knew I Loved You	4:07	Rock	1999-10-23	2000-01-29	1	71.0	1999-10-23
3	2000	Madonna	Music	3:45	Rock	2000-08-12	2000-09-16	1	41.0	2000-08-12
4	2000	Aguilera, Christina	Come On Over Baby (All I Want Is You)	3:38	Rock	2000-08-05	2000-10-14	1	57.0	2000-08-05

Next steps: Generate code with df_tidy | View recommended plots | New interactive sheet

5. Split into Two Tables

Why split? We often separate the **static** song info (e.g., artist, track, time, genre) from the **weekly** chart performance (week, rank, date).

- **Songs Table**: Contains unique identifiers for each song plus basic metadata.
- **Positions Table**: Contains `(song_id, week, rank, date)`, referencing the **song_id** from the songs table.

Generate a new `song_id` for each unique song, then merge/join this `song_id` back into the melted DataFrame.

```
# 5.1 Create a songs table (unique rows). Assign a song_id.
# Then merge that ID back into the main tidy DataFrame.
songs_table = df_tidy.drop_duplicates(subset=['artist.inverted', 'track']).copy()
songs_table['song_id'] = range(1, len(songs_table) + 1)
songs_table = songs_table[['song_id', 'artist.inverted', 'track', 'time', 'year']]
songs_table.rename(columns={'artist.inverted': 'artist'}, inplace=True)

print("Songs Table:")
print(songs_table.head())
```

```
Songs Table:
   song_id              artist                                track  time  \
0        1     Destiny's Child              Independent Women Part I  3:38
1        2             Santana                         Maria, Maria  4:18
2        3       Savage Garden                I Knew I Loved You  4:07
3        4             Madonna                                Music  3:45
4        5  Aguilera, Christina  Come On Over Baby (All I Want Is You)  3:38

   year
0  2000
1  2000
2  2000
3  2000
4  2000
```

6. Create the Positions Table

Once the `song_id` is part of the melted DataFrame, we can select the columns `(song_id, week, rank, date)` into a separate DataFrame called **positions**.

```
# Your code here
# Your code here
positions_table = pd.merge(df_tidy, songs_table[['song_id', 'artist', 'track']],
                     left_on=['artist.inverted', 'track'], right_on=['artist', 'track'], how='left')
positions_table = positions_table[['song_id', 'week', 'rank', 'date']]

print("Positions Table:")
print(positions_table.head())
```

```
Positions Table:
   song_id  week  rank       date
0        1     1  78.0 2000-09-23
1        2     1  15.0 2000-02-12
2        3     1  71.0 1999-10-23
3        4     1  41.0 2000-08-12
4        5     1  57.0 2000-08-05
```

By the end of this process you should have:

- A **songs** table with unique song info and a `song_id` for each song (keep song names, artists, etc.)
- A **positions** table with `(song_id, week, rank, date)` for each song-week.

  For example:

  songs table:

  | song_id | artist | track | time | year | |---------|--------------|--------------|------|-----| | 1 | The Beatles | Hey Jude | 3:35 | 1968 | | 2 | Elton John | Rocket | 3:10 | 1972 | | 3 | Whitney Houston | I Will Always | 4:30 | 1992 | | ... | ... | ... | ... | ... |

positions table:

song_id	week	rank	date
1	1	1	1968-09-14
1	2	1	1968-09-21
1	3	1	1968-09-28
...	...	...	...

7. Save Tidy Data to Feather

We want to save:

- The **complete tidy** DataFrame (one row per song-week) with a filename suffix `_tidy`.
- The **songs** and **positions** tables as separate Feather files.

```
# Your code here

df_tidy.reset_index(drop=True).to_feather('df_tidy.feather')
songs_table.reset_index(drop=True).to_feather('songs_table.feather')
positions_table.reset_index(drop=True).to_feather('positions_table.feather')

print("Data has been saved in the Feather format.")
```

```
Data has been saved in the Feather format.
```

Part 1 - Basic Analysis

Below are some guided analyses to help you get started. Feel free to explore more on your own!

A. Only songs that reached top 10

1. Use `query()` to filter rows in `positions` for `rank <= 10`.
2. Merge or join with the `songs` table to get the song details.

Hint: Remember you can drop duplicates if you only want the list of unique songs that reached top 10.

```
# Your code here

pos_top_ten = positions_table.query('rank <= 10')
weeks_in_top_ten = pos_top_ten.groupby('song_id').size().reset_index(name='weeks_in_top_ten')
song_details_top_ten = pd.merge(weeks_in_top_ten, songs_table, on='song_id', how='left')
unique_songs_top_ten = song_details_top_ten.drop_duplicates(subset=['song_id'])

print("Songs that Reached Top ten:")
print(unique_songs_top_ten[['artist', 'track', 'time', 'year']].drop_duplicates())
unique_songs_total = unique_songs_top_ten['song_id'].nunique()
print("Total number of unique songs that reached the top ten:", unique_songs_total)
```

```
51              Spears, Britney                Oops!.. I Did It Again  3:30
52         Backstreet Boys, The                     Shape Of My Heart  3:49

    year
0   2000
1   2000
2   2000
3   2000
4   2000
5   2000
6   2000
7   2000
8   2000
9   2000
10  2000
11  2000
12  2000
13  2000
14  2000
15  2000
16  2000
17  2000
18  2000
19  2000
20  2000
21  2000
22  2000
23  2000
24  2000
25  2000
26  2000
27  2000
28  2000
29  2000
30  2000
31  2000
32  2000
33  2000
34  2000
35  2000
36  2000
37  2000
38  2000
39  2000
40  2000
41  2000
42  2000
43  2000
44  2000
45  2000
46  2000
47  2000
48  2000
49  2000
50  2000
51  2000
52  2000
Total number of unique songs that reached the top ten: 53
```

B. How long did each song stay in the top 10?

1. Filter `positions` for `rank <= 10`.
2. Group by `song_id` and count the number of rows. This gives `weeks_in_top_ten`.
3. Merge back with the `songs` table to display the track/artist.

```
# Your code here
print("Songs and their weeks: Top ten")
print(song_details_top_ten[['artist', 'track', 'weeks_in_top_ten']])
```

```
50                     Sonique                      It Feels So Good
51               Spears, Britney                Oops!.. I Did It Again
52          Backstreet Boys, The                     Shape Of My Heart

    weeks_in_top_ten
0                 17
1                 18
2                 17
3                 12
4                  8
5                 12
6                 11
7                 10
8                 13
9                 12
10                11
11                 7
12                19
13                18
14                15
15                14
16                 4
17                19
18                13
19                11
20                 7
21                 6
22                 5
23                16
24                18
25                13
26                 4
27                17
28                16
29                12
30                12
31                 7
32                 9
33                 9
34                 9
35                10
36                 9
37                 9
38                 6
39                 8
40                 7
41                 9
42                 5
43                 4
44                 4
45                 7
46                 6
47                 5
48                 3
49                 2
50                 3
51                 3
52                 3
```

C. In which week did each song *first* reach the top 10?

1. Again filter for `rank <= 10`.
2. Group by `song_id` and find the minimum `week` value. Call this `week_reached_top_10`.

```
# Your code here
first_week_top_ten = pos_top_ten.groupby('song_id')['week'].min().reset_index()
first_week_top_ten.rename(columns={'week': 'week_reached_top_ten'}, inplace=True)
first_songs_top_ten = pd.merge(first_week_top_ten, songs_table, on='song_id', how='left')
print("Song First Reached Week Each Top ten:")
print(first_songs_top_ten[['artist', 'track', 'week_reached_top_ten']])
```

```
Song First Reached Week Each Top ten:
                        artist                                track  \
0           Destiny's Child              Independent Women Part I
1                   Santana                          Maria, Maria
```

```
2                 Savage Garden                   I Knew I Loved You
3                       Madonna                                Music
4             Aguilera, Christina  Come On Over Baby (All I Want Is You)
5                          Janet                Doesn't Really Matter
6                 Destiny's Child                         Say My Name
7               Iglesias, Enrique                         Be With You
8                           Sisqo                          Incomplete
9                        Lonestar                              Amazed
10                        N'Sync                    It's Gonna Be Me
11            Aguilera, Christina                   What A Girl Wants
12               Vertical Horizon                  Everything You Want
13                          Creed                  With Arms Wide Open
14                        Aaliyah                          Try Again
15                matchbox twenty                               Bent
16                  Carey, Mariah                Thank God I Found You
17                    Hill, Faith                             Breathe
18                            Mya      Case Of The Ex (Whatcha Gonna Do)
19                          Dream                     He Loves U Not
20                  Braxton, Toni                  He Wasn't Man Enough
21                  Anthony, Marc                     You Sang To Me
22                            98°     Give Me Just One Night (Una Noche)
23                Destiny's Child                     Jumpin' Jumpin'
24                   3 Doors Down                          Kryptonite
25                          Sisqo                          Thong Song
26            Aguilera, Christina                       I Turn To You
27               Houston, Whitney                  My Love Is Your Love
28                             Joe                       I Wanna Know
29                Jordan, Montell                   Get It On.. Tonite
30                          N'Sync                      Bye Bye Bye
31                          Eminem                  The Real Slim Shady
32                            Pink                          Most Girls
33                Mumba, Samantha                     Gotta Tell You
34                          Blaque                   Bring It All To Me
35  Elliott, Missy "Misdemeanor"                           Hot Boyz
36                    Gray, Macy                           I Try
37                          N'Sync                  This I Promise You
38                       Ruff Endz                           No More
39           Backstreet Boys, The    Show Me The Meaning Of Being Lonely
40                       Nine Days           Absolutely (Story Of A Girl)
41                    Dion, Celine                That's The Way It Is
42                       Blink-182                All The Small Things
43                       Eiffel 65                    Blue (Da Ba Dee)
44                    Badu, Erkyah                          Bag Lady
45                            Pink                         There U Go
46                          Creed                             Higher
47                          Nelly            (Hot S**t) Country Grammar
48                            Next                             Wifey
49                       Kenny G     Auld Lang Syne (The Millenium Mix)
50                       Sonique                    It Feels So Good
51                Spears, Britney                Oops!.. I Did It Again
52           Backstreet Boys, The                     Shape Of My Heart

    week_reached_top_ten
0                      7
```

Part 2 - Visualization

For this part, you may use **Altair** (*Preferable!*), **Matplotlib**, **Seaborn**, or any other Python visualization library.

Below are some suggested questions and plots:

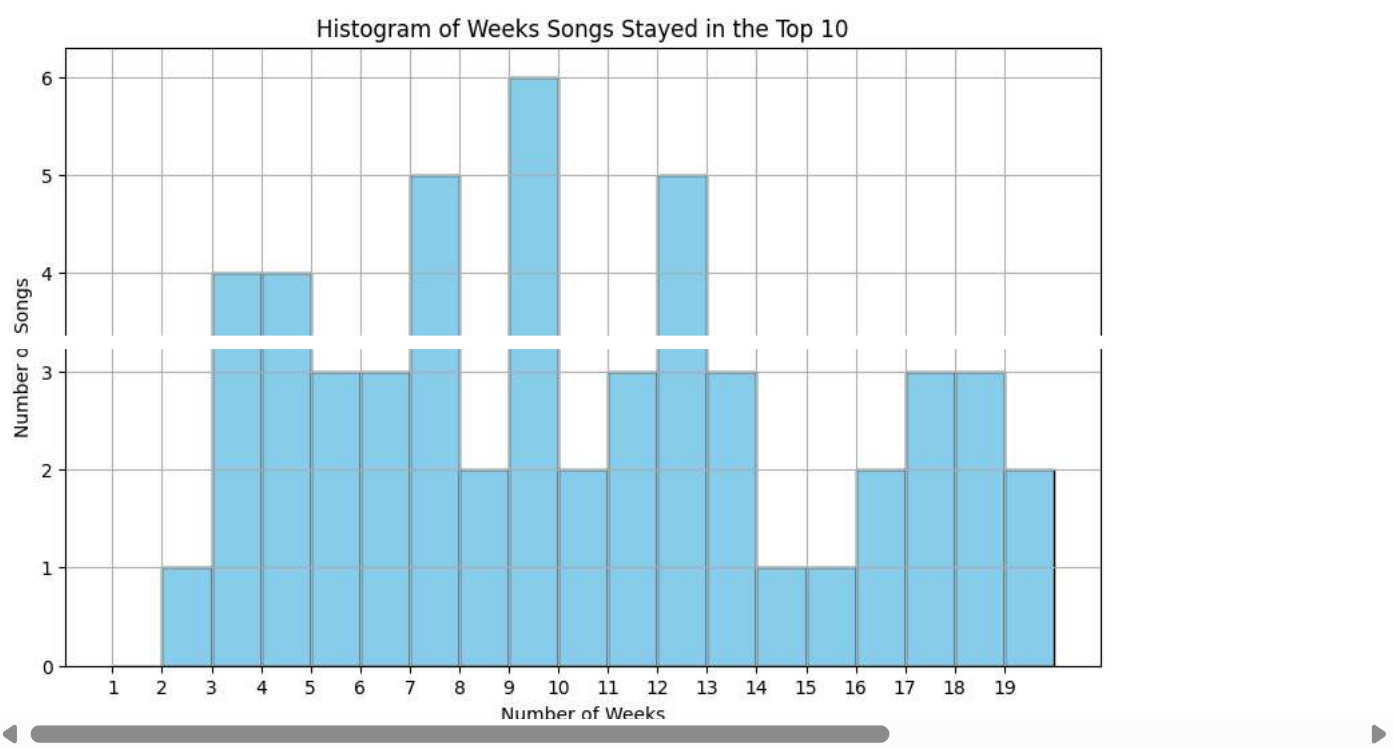
1. **Histogram of Weeks in the Top 10**
    - How many songs stayed for 1 week vs. 2 weeks vs. 10 weeks in the top 10?
    - Plot this distribution.

```
# 1. Example: Histogram of the number of weeks each song stayed in the top 10.
# (Assuming you've created a 'weeks_in_top_ten' table or series.)

# Your code here
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.hist(song_details_top_ten['weeks_in_top_ten'], bins=range(1, song_details_top_ten['weeks_in_top_ten'].max() + 2), color='skyblue', edgec
plt.title('Histogram of Weeks Songs Stayed in the Top 10')
plt.xlabel('Number of Weeks')
plt.ylabel('Number of Songs')
plt.xticks(range(1, song_details_top_ten['weeks_in_top_ten'].max() + 1))
```

```
plt.grid(True)
plt.show()
```



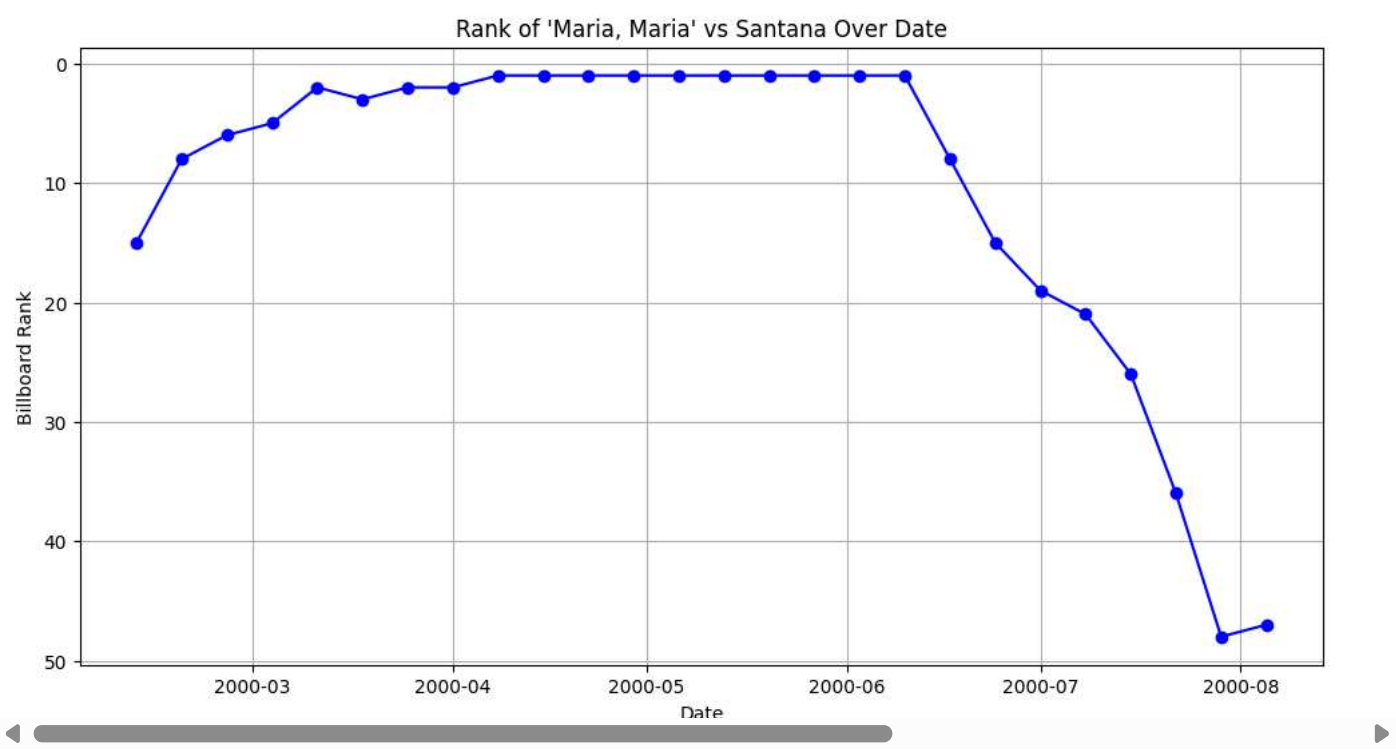
2. **Line Chart of a Song's Rank Over Time**
   - Pick a well-known song from the dataset (e.g., one of the most weeks at #1).
   - Plot its `rank` vs. `date` (or `week`).
   - The line should show how the rank changes over consecutive weeks.

```
# Use the original `positions` DataFrame to plot a song's rank over time.
# Your code here
positions_mergerd = pd.merge(positions_table, songs_table[['song_id', 'artist', 'track']], on='song_id', how='left')
song_title = "Maria, Maria"
song_artist = "Santana"

song_info = positions_mergerd[(positions_mergerd['track'] == song_title) &
                              (positions_mergerd['artist'] == song_artist)]

song_info = song_info.sort_values('date')


plt.figure(figsize=(12, 6))
plt.plot(song_info['date'], song_info['rank'], marker='o', linestyle='-', color='b')
plt.title(f"Rank of '{song_title}' vs {song_artist} Over Date")
plt.xlabel('Date')
plt.ylabel('Billboard Rank')
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```



3. **Distribution of Peak Ranks**
    - For each song, find its **best** (lowest number) rank.
    - Plot a histogram of these peak positions. Where do most songs peak? (#1, #5, #20, etc.)

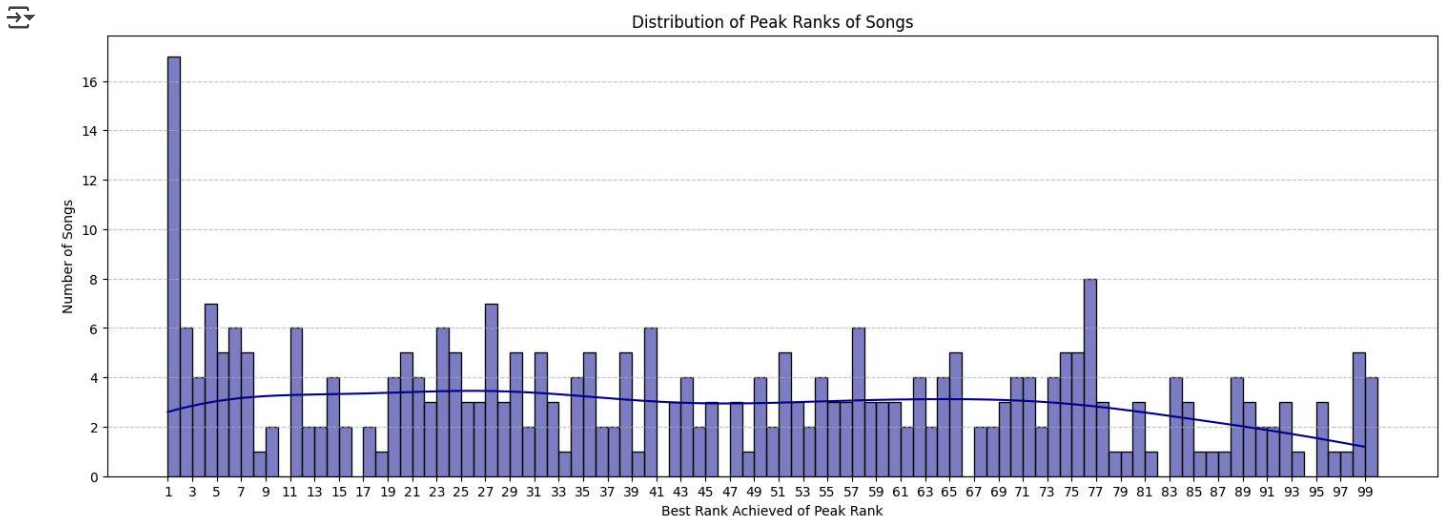
```
# Your code here
import seaborn as sns
rank_of_peaks = positions_table.groupby('song_id')['rank'].min().reset_index()
rank_of_peaks.rename(columns={'rank': 'peak_rank'}, inplace=True)

rank_of_peaks = pd.merge(rank_of_peaks, songs_table[['song_id', 'artist', 'track']], on='song_id', how='left')

rank_of_peaks['peak_rank'] = rank_of_peaks['peak_rank'].astype(int)

plt.figure(figsize=(18, 6))
sns.histplot(rank_of_peaks['peak_rank'], bins=range(1, int(rank_of_peaks['peak_rank'].max()) + 2), kde=True, color="darkblue")

plt.title('Distribution of Peak Ranks of Songs')
plt.xlabel('Best Rank Achieved of Peak Rank')
plt.ylabel('Number of Songs')
plt.xticks(range(1, int(rank_of_peaks['peak_rank'].max()) + 1, 2), rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Bonus: Additional Analysis (Optional, but will give you extra credit!)

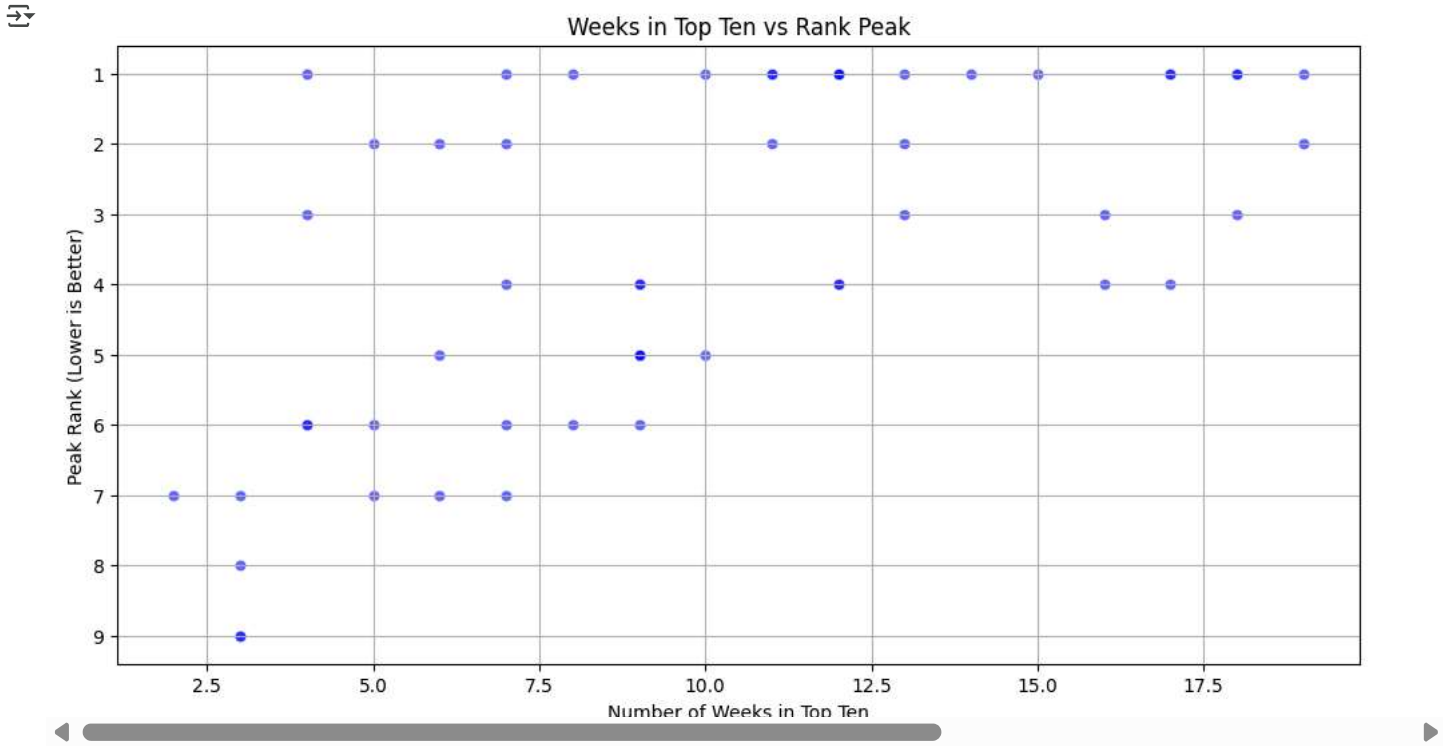
(Add more charts and visualizations as you see fit!)

Be creative and explore the dataset! showcase at least a new plot or analysis.

```
# Your extra analyses here

plt.figure(figsize=(12, 6))
sns.scatterplot(x=song_details_top_ten['weeks_in_top_ten'], y=rank_of_peaks['peak_rank'], alpha=0.6, color='blue')

plt.title("Weeks in Top Ten vs Rank Peak")
plt.xlabel("Number of Weeks in Top Ten")
plt.ylabel("Peak Rank (Lower is Better)")
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```



```
plt.figure(figsize=(14, 6))
sns.boxplot(x=positions_table['week'], y=positions_table['rank'])

plt.title("Rank Distribution Over Weeks")
plt.xlabel("Week")
plt.ylabel("Rank")
plt.gca().invert_yaxis()
plt.grid(True)
plt.show()
```

